

Laboratory 01

Conceptual Communication System and Signal Flow Analysis

ITAI 4370: AI in 5G and 6G Communications and ORAN Networks

Monica Joya

June 8, 2026

Objective

This lab has two parts. Part 1 is a diagram of a communication system, drawn in a free tool. Part 2 uses Python and matplotlib to show the signal at each step as it moves through that system.

Part 1: Conceptual Diagram

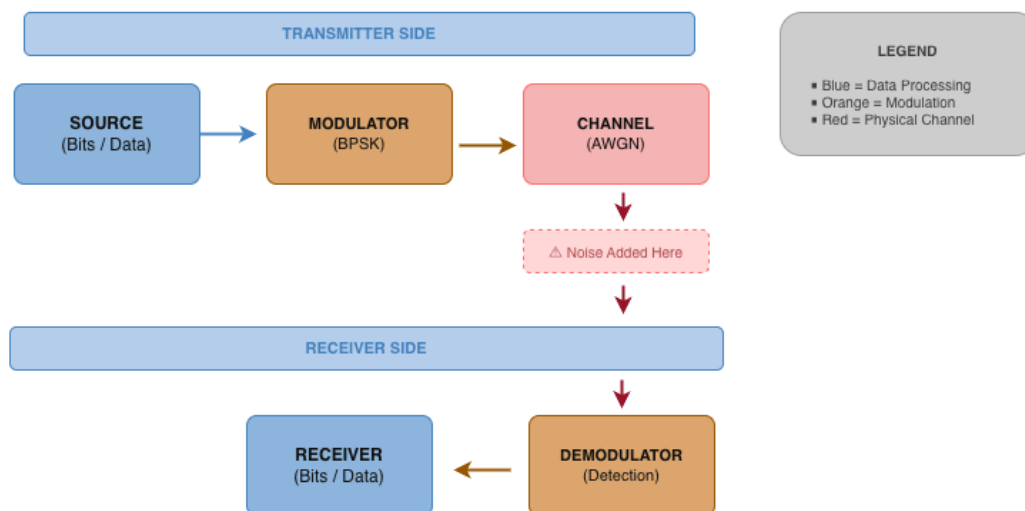


Figure 1. The communication system made in draw.io

My diagram shows the path a message takes from one end of a communication system to the other. It runs left to right across the top, then drops down and runs back along the bottom: Source, Modulator, Channel, then Demodulator and Receiver.

The Source produces the bits. The Modulator puts those bits onto a carrier wave. The Channel is the medium the wave travels through, and noise is added there. The Demodulator reads the wave back into bits, and the Receiver is where they arrive. The colors sort the blocks by job: blue for the points where data starts and ends, orange for the two modulation steps, and red for the channel.

Part 2: Signal Flow Analysis in Python

My Notebook follows one set of bits through the system using BPSK, or Binary Phase Shift Keying. BPSK is the simplest way to put bits on a wave. It sets the phase of the wave, which is just which way up the wave sits. A 1 leaves the wave one way and a 0 flips it the other.

Stage 1, Source. The message is a string of random 1s and 0s. Each bit is held for one bit period so it lines up with the wave.

Stage 2, Modulator. Each bit becomes a level, +1 for a 1 and -1 for a 0. That level multiplies a steady carrier wave. The +1 leaves the wave alone, and the -1 turns it upside down. The bit rides on which way the wave points.

Stage 3, Channel. Random noise is added to the wave. The amount is set by a signal to noise ratio of 10 dB, so the wave stays well above the noise. The plot shows the noisy wave with the clean one behind it, so the effect of the channel is easy to see.

Stage 4, Demodulator and Receiver. The noisy wave is multiplied by the same carrier again. The parts that still match the carrier add up to a positive number, and the flipped parts add up to a negative one. For each bit period the code adds up the result, then reads positive as a 1 and negative as a 0.

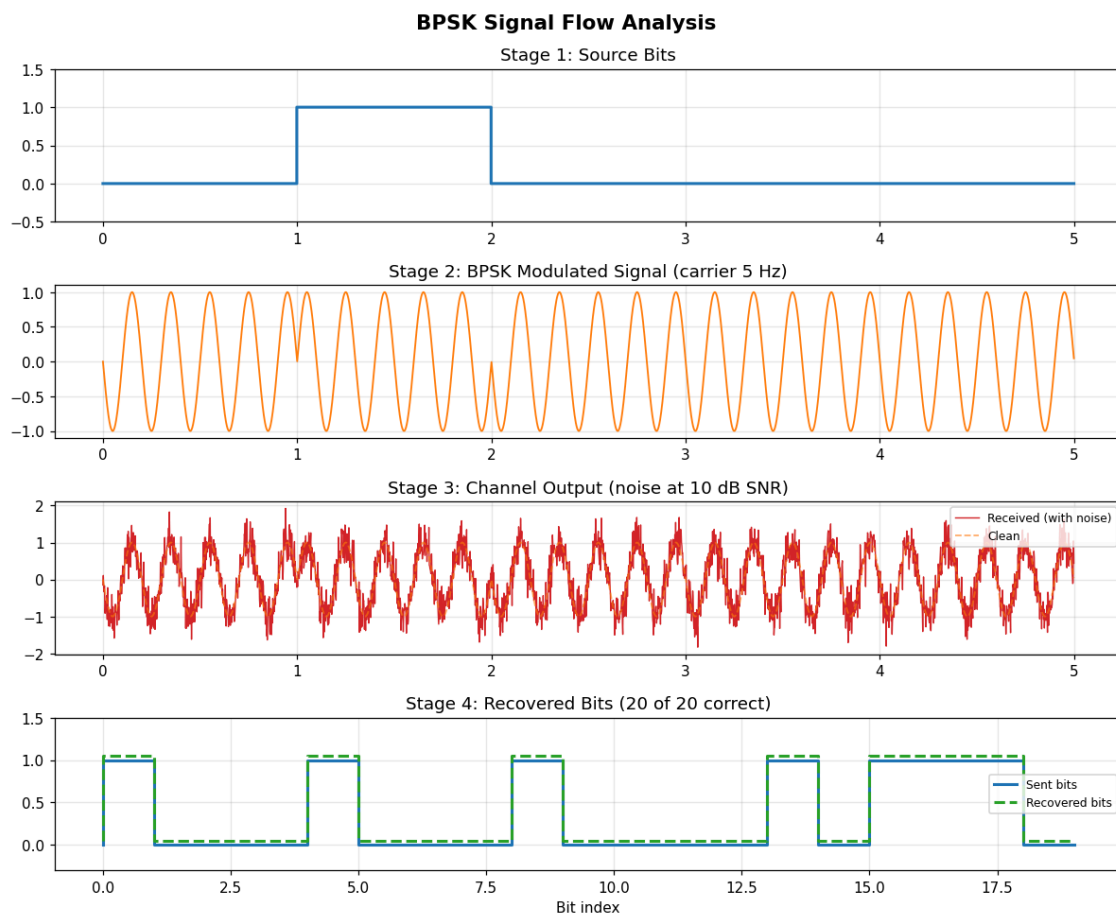


Figure 2. The signal at each stage.

Result. At a 10 dB signal to noise ratio, all 20 bits came back correct, so the message made it through the noise unchanged.

Python Code

The full script is below. It is also turned in as a runnable Jupyter notebook.

```
import matplotlib.pyplot as plt
import numpy as np

# Used a fixed seed so the random bits and noise are the same every run
np.random.seed(42)

# Parameters
num_bits = 20 # number of bits in the message
fc = 5 # carrier frequency in Hz
fs = 500 # samples per bit, controls how smooth the wave looks
t = np.linspace(0, num_bits, num_bits * fs)

# Stage 1: Source. Creates the random message bits.
bits = np.random.randint(0, 2, num_bits)

# Stretches each bit so it lasts one full bit period (for the plot)
bits_signal = np.repeat(bits, fs)

# Stage 2: Modulator. Maps bits to levels, then multiplies by a carrier wave.
symbols = 2 * bits_signal - 1 # 1 becomes +1, 0 becomes -1
carrier = np.sin(2 * np.pi * fc * t)
modulated = symbols * carrier # a -1 flips the wave 180 degrees (BPSK)

# Stage 3: Channel. Adds random noise scaled to a 10 dB signal to noise ratio.
snr_db = 10
snr_linear = 10 ** (snr_db / 10)
noise_power = 1 / snr_linear
noise = np.sqrt(noise_power) * np.random.randn(len(modulated))
received = modulated + noise

# Stage 4: Demodulator and Receiver. Multiplies by the carrier, then decides each bit.
demod = received * carrier

recovered_bits = []
for i in range(num_bits):
    segment = demod[i*fs:(i+1)*fs]
    recovered_bits.append(1 if np.sum(segment) > 0 else 0)
recovered_bits = np.array(recovered_bits)

errors = int(np.sum(bits != recovered_bits))
print(f"{num_bits - errors} of {num_bits} bits recovered correctly ({errors} errors)")

# Plots the signal at each stage of the flow
window = 5 * fs # shows the first five bits for the wave plots
t_win = t[:window]

fig, axes = plt.subplots(4, 1, figsize=(11, 9))
fig.suptitle("BPSK Signal Flow Analysis", fontsize=14, fontweight="bold")

# Stage 1
axes[0].step(t_win, bits_signal[:window], color="#1f77b4", linewidth=2)
axes[0].set_title("Stage 1: Source Bits")
axes[0].set_ylim(-0.5, 1.5)
axes[0].grid(True, alpha=0.3)

# Stage 2
```

```

axes[1].plot(t_win, modulated[:window], color="#ff7f0e", linewidth=1.2)
axes[1].set_title(f"Stage 2: BPSK Modulated Signal (carrier {fc} Hz)")
axes[1].grid(True, alpha=0.3)

# Stage 3
axes[2].plot(t_win, received[:window], color="#d62728", linewidth=0.9, label="Received (with noise)")
axes[2].plot(t_win, modulated[:window], color="#ff7f0e", linewidth=1.2, linestyle="--", alpha=0.6, label="Clean")
axes[2].set_title(f"Stage 3: Channel Output (noise at {snr_db} dB SNR)")
axes[2].legend(fontsize=8)
axes[2].grid(True, alpha=0.3)

# Stage 4
axes[3].step(range(num_bits), bits, color="#1f77b4", linewidth=2, label="Sent bits")
axes[3].step(range(num_bits), recovered_bits + 0.05, color="#2ca02c", linewidth=2, linestyle="--", label="Recovered bits")
axes[3].set_title(f"Stage 4: Recovered Bits ({num_bits} - errors) of {num_bits} correct")
axes[3].set_ylim(-0.3, 1.5)
axes[3].set_xlabel("Bit index")
axes[3].legend(fontsize=8)
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Conclusion

The two parts fit together. Part 1 (diagram) shows the shape of the system, the blocks and the order they run in. Part 2 (ipynb lab) puts that system to work in code and follows one message as it passes through. Seeing the bits come out the way they went in is what a signal flow analysis is meant to show, from the source all the way to the receiver.